Magpie — Sprint 18 spec: fee correctness (2026 AU model) + buy calculator v1

Audience: Codex. **Author:** Claude. **Status: ready to implement. Locks:** D-128…D-131 (+ D-121/D-129 evidence rules). **Facts:** F-19 (eBay AU fee restructure, verified 2026-06-16), F-20 (API Pro-gating — see §8). **This opens the post-roadmap "profit engine" chapter: profit is made at acquisition, so the first deliverables are correct fee maths + a point-of-purchase Max-Buy tool.**

---

1. Goal

Two deliverables that stand together: **(A)** Make Magpie's fee model **true for eBay AU as of 12 May 2026** (F-19) — seller modes, the buyer-paid Buyer Protection fee (BPF), and **normalisation between buyer-visible and seller-receives prices** so comps and margins stop overstating. **(B)** Ship the **buy calculator v1** — a **mobile-first reverse-margin tool** answering, at the point of purchase: *"given this sell price, what's the MOST I can safely pay?"* → **Max Buy Price → BUY / PASS / MAYBE → expected net profit → ROI → confidence/source basis.**

All-local. No new eBay API calls (see §8). No AI numbers, no scraping (D-121/D-129).

2. Scope

IN:

- **A — Fee engine 2026:** seller-mode setting; BPF maths (both directions); updated `FeeSchedule`; comp price-basis + normalisation (§3–§4).
- **B — Buy calculator v1:** reverse-margin computation + verdict + a fast phone UI; own-sales evidence lookup; transient what-if input; auction max-bid mode (§5–§6).

OUT (explicitly):

- ❌ The **evidence bank** proper (Sprint 19) — v1's evidence lookup is own `SaleRecord`s (exact-then-similar, reusing the Sprint-14 matching) + optionally existing approved `Comparable`s if cheap; the fuller comp-memory UX comes next sprint.

- ❌ Any **AI-produced price** (D-121/D-129) or **live marketplace fetching/scraping** (F-09). The calculator consumes owned evidence and human inputs only.
- ❌ Any **new eBay-API-dependent feature** (F-20 gate — plan status unknown; §8).
- ❌ Risk checklist (v1.1, next), lot manager, profit-intel views (later sprints per D-131).

3. Part A — fee engine for the 2026 AU model (F-19)

- **Seller-mode setting** (in Settings, per-user single value): `free_selling` (≤$25k, no FVF; buyers pay BPF; international-delivery fee 3%) · `pro_starter` (13.4% incl. GST FVF; no BPF on listings) · `pro_other` (configurable FVF %, for Pro Basic+ tiers) · `legacy/manual` (keep a configurable generic mode for non-eBay channels). Default for Regan: `free_selling` (confirm at deploy).
- **BPF function, exact and tested (piecewise, GST-inclusive):** `BPF(p) = 0.30 + 8%·min(p,20) + 6%·max(0,min(p,500)−20) + 4%·max(0,min(p,5000)−500)`; the portion of `p` above $5,000 adds nothing. Implement **both directions**: `buyer_total(p) = p + BPF(p)` and the **inverse** (given a buyer-visible total, recover the seller price — piecewise-linear, exactly invertible; unit-test round-trips at the breakpoints $20/$500/$5,000 and the cap).
- **`FeeSchedule` update:** replace the obsolete seller-FVF placeholders with the 2026 structures per mode. Keep it data/config, not hardcoded, so future fee changes are edits not code.
- **Comp/sale price basis:** add a `price_basis` field (`buyer_visible` | `seller_receives` | `unknown`) to `Comparable` (additive migration). Capture flow lets the human tag it — ***(amended v1.1)*** **default `unknown` for all eBay AU sold captures**, since sold results don't reliably reveal whether the seller was Pro (no BPF) or casual (BPF included); the user may positively set `buyer_visible` / `seller_receives` when they can tell. Provide a **normalisation utility** used by all grids/aggregates: convert known-basis rows to a single canonical basis (**seller_receives**) before computing low/median/high. Grids display which basis is shown. **`unknown` rows are included but carry a visible "basis-uncertain" label** (honest-uncertainty pattern) — never silently treated as a known basis.
- **Own `SaleRecord`s** already store actual amounts received — treat as `seller_receives`; no change beyond labelling.

4. Where this shows up

- Pricing grids + own-sales views: aggregate on the canonical basis; label it; per-row basis visible on drill-through.
- Sales/profit maths anywhere fees are applied: route through the new fee engine by seller mode.
- Honest states: if a grid mixes bases or has `unknown` rows, say so plainly rather than implying precision.

5. Part B — buy calculator v1 (reverse-margin, D-130)

Inputs (fast, few, finger-sized):

1. **Expected sell price** — from evidence when available (§6) or typed as a **what-if** (clearly labelled "your estimate — not evidence", never persisted as valuation data, D-129).
2. **Channel/mode** — defaults to the seller-mode setting; selectable (e.g. eBay free-selling vs a non-eBay cash sale with zero fees).
3. **Cost assumptions** — postage/label estimate, packaging (prefill from small per-category defaults; editable), optional refurb.
4. **Required profit** — target ROI % or flat $ minimum (persistent user setting; editable per calc).
5. Optional: the **asking price** (what the seller wants) → enables the verdict.

Computation: `net_proceeds = sell_price(seller_receives basis) − fees(mode) − postage − packaging − refurb`. For a **flat-$ target**: **`Max Buy = net_proceeds − required_profit`**. For an **ROI target**, ROI has an explicit, UI-labelled denominator with two modes *(amended v1.1 per ChatGPT review)*:

- **All-in cash (DEFAULT):** ROI measured against all cash in the deal (buy + refurb + packaging + postage/label). **`MaxBuy = (seller_receives − fees) / (1 + ROI) − non_buy_costs`** where `non_buy_costs = refurb + packaging + postage`. *(Fees scale with the sell price so they sit inside the division; under free_selling fees = $0 and this reduces to `seller_receives/(1+ROI) − non_buy_costs`.)*
- **On buy price:** ROI against the purchase price alone. **`MaxBuy = net_proceeds / (1 + ROI)`**. The UI always states which basis is active ("ROI on all-in cash" / "ROI on buy price"); both formulas unit-tested, including their equivalence check under zero fees/zero non-buy costs. **Verdict vs asking:** `BUY` (asking ≤ MaxBuy), `MAYBE` (within a small configurable band above), `PASS` (above the band). **Outputs, in order (D-**

130): Max Buy Price (the headline) → **BUY/PASS/MAYBE** → expected **net profit** at the asking price → **ROI** → **confidence/source basis** (`own sale — exact` / `own sale — similar` / `approved comp` / `what-if (your estimate)`). **Auction mode:** same maths, headline reframed as **Max Bid**; one toggle, no separate screen.

6. Evidence lookup (v1 slice of D-128)

- On identifying/naming the item (manual entry or picking an existing item/category), the calculator **checks owned evidence first**: exact-item `SaleRecord` s, then similar (reuse the Sprint-14 own-sales matching), then existing approved `Comparable` s if inexpensive to include. If found → prefill the sell price from it (median of the evidence, **computed only from in-Magpie rows**, D-129) and set the confidence label accordingly.
- If none → show the **sold-search links** (existing Sprint-13/14 panel) + the what-if input. Capturing a comp from those searches uses the **existing capture flow** (now with price-basis tagging) — every capture makes the next lookup better. **No new scraping/fetching of any kind.**

7. Design

Mobile-first is the point — this is used standing at a market stall over Tailscale. One screen, big touch targets, the **Max Buy figure huge**, verdict colour-coded (with a colour-blind-safe distinction), Sprint-16 bright high-contrast tokens. Latency target: instant math on-device/on-request; no long spinners. Desktop layout can be the same component, wider.

8. F-20 gate (read this, Codex)

eBay AU has made **API/third-party integrations Pro-Basic-plan-gated** (F-20). Regan's plan status is **unknown**. Therefore: **this sprint must not add any new eBay API calls** and must not assume publish/order-sync availability. Everything here is local maths + owned data. (Existing integration code is untouched; if it starts failing due to plan gating, that's a separate decision, not this sprint.)

9. Acceptance criteria (closure gate)

- **BPF maths:** unit tests cover the formula, the tier breakpoints ($20/$500/$5,000), the >$5,000 cap, and **exact round-trip inversion** (seller ⇄ buyer-visible).
- **Seller modes:** fee outcomes correct per mode (free_selling: $0 FVF + BPF-on-

buyer; pro_starter: 13.4% no BPF; configurable pro_other; manual).

- **Normalisation:** grids/aggregates compute on the canonical basis from known-basis rows; `price_basis` captured on new comps with `unknown` **as the eBay AU default**; unknown rows included but visibly labelled basis-uncertain; **no fabricated precision**.
- **Calculator:** Max Buy / verdict / profit / ROI correct for flat-$ **and both ROI bases (all-in cash default, on-buy-price optional; equivalence under zero fees/costs tested; UI labels the active basis)**; auction toggle reframes correctly; **what-if inputs are never persisted as evidence** (test-proven); evidence prefill uses **only in-Magpie rows** (test or CI guard per D-129); confidence label matches the source.
- **No new network calls** to eBay or anywhere (assert; F-20/F-09).
- **Mobile:** phone screenshots of the calculator in use are headline evidence; no overflow at ~390px; big targets.
- **Process:** backend + frontend tests green; **dual-lane Validation green**; additive migrations only; **pre-migration encrypted backup + restore spot-check** (new fields); rebuild + restart; evidence as `SPRINT18_EVIDENCE.md`.

10. For Codex

Reasoning: High. The fee engine is **data-driven, exactly tested, and invertible** — the BPF breakpoints are where bugs will hide. The calculator is **reverse-margin only**: it never asserts what an item is worth; it converts a human/evidence sell price into a Max Buy. **Hard lines:** no AI-produced numbers anywhere in the math path (D-129 — guard it); no new marketplace fetching (F-09) and **no new eBay API usage** (F-20 gate); what-ifs never persist as evidence; additive migrations; never print/commit secrets (`.env` values, keys, passphrases — note a rotation is in progress, so don't echo current values in any diagnostic). Evidence as `SPRINT18_EVIDENCE.md`. This is the first sprint of the profit-engine chapter (D-131: 18 fees+calculator → 19 evidence bank → 20 profit intel → 21 lot manager → 22 multi-channel pack).